

Person 1: Backend Engineer (Gateway Middleware & Interception)

Focus: Sub-Story 3.1a & 3.1b — Request interception, payload extraction, early 202 responses, and client timeout detection.

Tools to Use: Java/Kotlin, Spring Boot, Spring Cloud Gateway (or `OncePerRequestFilter`), and Servlet/WebFlux connection listeners.

How to Build:

- **Targeted Interceptor:** Construct a dedicated Gateway Filter designed specifically to listen for traffic destined for flagged "High Risk" endpoints (e.g., `/api/credit/approve`).
- **Payload Parking Mechanism:** Read the incoming request body, headers, target URL, and `agent_id` from the AI agent. Completely halt the routing chain so the request never reaches its intended destination. Pass this data to the database service to be persisted.
- **Agent Acknowledgment:** Construct and return a 202 Accepted (Pending) HTTP response back to the originating AI agent, signaling that the action is safely held for human review.
- **Timeout Catch (Unhappy Path):** Implement a connection state listener. If the AI agent terminates the network connection before the Gateway can deliver the 202 response, intercept the resulting `ClientAbortException`. Trigger an immediate database update changing the parked payload's status to `Delivery_Timeout_Warning`.

Person 2: Database Engineer (Persistence & Core API)

Focus: Sub-Story 3.1a, 3.1b, & 3.2a — Quarantine schema creation, state machine updates, and exact payload forwarding logic.

Tools to Use: PostgreSQL, Spring Data JPA / Hibernate, Jackson JSON, and a Java HTTP client (e.g., `RestTemplate` or `WebClient`).

How to Build:

- **Quarantine Schema:** Execute DDL migrations to create the PostgreSQL quarantine table containing columns for `quarantine_id`, `endpoint`, `payload (JSONB)`, `agent_id`, and `status`.
- **State Management Endpoints:** Build the REST controllers that the React frontend will consume: `GET /quarantine`, `POST /quarantine/{id}/approve`, and `POST /quarantine/{id}/reject`.
- **Discard Logic:** When the `/reject` endpoint is hit, update the specific database row's status to `Discarded` and permanently remove it from the active review queue.
- **Execution Logic:** When the `/approve` endpoint is hit, query the exact original JSON payload and target endpoint from the database. Use an internal HTTP client to fire that precise payload directly to its intended destination, bypassing the middleware filter. Update the status to `Approved` upon successful execution.

Person 3: Frontend Engineer (HITL Dashboard UI)

Focus: Sub-Story 3.1a & 3.2a — Compliance officer interface, real-time queueing, and approval/rejection workflows.

Tools to Use: React, TypeScript, a state management library (Context API/Zustand), and WebSocket/SSE hooks.

How to Build:

- **Real-Time Queue Dashboard:** Build a secure view for Compliance Officers that displays a live list of parked payloads. Connect this view to the real-time notification stream so new interceptions appear instantly without a page refresh.
- **Timeout Alert Badges:** Implement conditional UI rendering to prominently flag any payload carrying the `Delivery_Timeout_Warning` status. Highlight these in red to alert the officer that the autonomous agent might attempt to duplicate the action.
- **Payload Review Modal:** Create a detail view where the officer can inspect the exact JSON payload the AI agent attempted to send, alongside the target endpoint and agent ID.
- **Action Execution:** Implement "Approve" and "Reject" buttons. Attach confirmation dialogue boxes to both actions to prevent accidental clicks. Wire these buttons to Person 2's backend POST endpoints and optimistically remove the item from the local UI queue upon a 200 OK response.

Person 4: DevOps & Security Engineer

Focus: Sub-Story 3.1b & 3.2a — Network timeouts, alerting infrastructure, and internal routing security.

Tools to Use: Terraform, Docker/Kubernetes, API Gateway configurations, and Webhook integrations (e.g., Slack, Microsoft Teams, or PagerDuty).

How to Build:

- **Timeout Simulation & Enforcement:** Configure strict connection read/write timeouts at the ingress layer (e.g., NGINX or AWS ALB) to accurately mimic and test network drops between the AI agents and the Spring Boot Gateway.
- **Active Officer Alerting:** Configure an external notification bridge. When a payload receives a `Delivery_Timeout_Warning` status, push an automated alert to the Compliance Officer's Slack or email channel to ensure immediate visibility of the unstable agent.
- **Internal Routing Security:** Establish network policies ensuring that the internal destination endpoints (the high-risk targets) strictly accept traffic *only* from the Gateway's execution service (Sub-Story 3.2a) and explicitly block any direct network attempts from the AI agents, enforcing the HITL architecture at the infrastructure level.

Here is a comprehensive test suite designed to validate the **Human-in-the-Loop (HITL) Architecture (Sub-Stories 3.1 & 3.2)**. This suite guarantees that high-risk payloads are successfully quarantined, network disconnections are handled gracefully, and compliance officers have full deterministic control over payload execution.

1. Backend Engineer (Gateway Middleware) Test Cases

Focus: Request interception, early HTTP 202 responses, and client disconnect handling.

Test ID	Test Case Description	Pre-conditions	Test Steps	Expected Result
TC - GW-01	High-Risk Endpoint Interception	Gateway running with /api/credit/approve flagged as High Risk.	AI agent sends a POST request with a JSON payload to /api/credit/approve.	Middleware intercepts the request, blocks it from routing to the destination, and initiates the quarantine database save.
TC - GW-02	Low-Risk Endpoint Bypass	Gateway running with /api/credit/status flagged as Low Risk.	AI agent sends a GET request to /api/credit/status.	Middleware allows the request to pass directly to the destination endpoint without parking it in the quarantine table.
TC - GW-03	202 Accepted (Pending) Response	AI agent initiates a High-Risk action.	Monitor the HTTP response returned to the AI agent.	Agent receives a 202 Accepted status with a payload indicating the request is pending human review.
TC - GW-04	Client Abort / Disconnect Handling	Agent sends a payload to a High-Risk endpoint.	Sever the agent's network connection <i>after</i> the payload is received	Gateway catches the ClientAbortException or equivalent network break, and triggers a state update to mark the parked

Test ID	Test Case Description	Pre-conditions	Test Steps	Expected Result
			but <i>before</i> the Gateway can send the 202 Accepted response.	payload as Delivery_Timeout_Warning.

2. Database Engineer (Persistence & Core API) Test Cases

Focus: Quarantine state machine, payload preservation, and execution routing.

Test ID	Test Case Description	Pre-conditions	Test Steps	Expected Result
TC - DB -01	Quarantine Persistence Integrity	Middleware intercepts a High-Risk payload.	Check the PostgreSQL quarantine table.	A new row exists containing the exact JSON payload, target endpoint, agent_id, and status = 'Pending'.
TC - DB -02	Rejection Workflow (/reject)	Payload ID 101 is parked in Pending state.	Send a POST /quarantine/101/reject request.	Database updates row 101 to status = 'Discarded'. The payload is <i>not</i> forwarded to the destination endpoint.
TC - DB -03	Approval Workflow & Exact Forwarding	Payload ID 102 is parked. Destination endpoint is mocked to record incoming requests.	Send a POST /quarantine/102/approve request.	Backend queries the exact original JSON payload from the DB and forwards it to the target endpoint. DB updates row 102 to status = 'Approved'.

Test ID	Test Case Description	Pre-conditions	Test Steps	Expected Result
TC - DB -04	Destination Down During Approval	Payload ID 103 is parked. The target internal destination endpoint is intentionally taken offline.	Send a POST /quarantine/103/approver request.	Forwarding fails. Backend catches the 502/503 error, rolls back the transaction, keeps the payload in Pending (or marks as Execution_Failed), and alerts the UI. Payload is not lost.

3. Frontend Engineer (HITL Dashboard UI) Test Cases

Focus: Real-time queue updates, visual alerting, and strict review mechanics.

Test ID	Test Case Description	Pre-conditions	Test Steps	Expected Result
TC - UI-01	Real-Time Dashboard Injection	Compliance officer is logged into the dashboard.	Mock the Gateway pushing a new quarantine notification via WebSocket/SE.	The new pending payload dynamically appears at the top of the officer's queue without requiring a page refresh.
TC - UI-02	Timeout Warning Badge Rendering	Dashboard receives a payload with status: 'Delivery_Timeout_Warning'.	View the queue UI.	The specific payload row is highlighted (e.g., red outline) and prominently displays a "Delivery Timeout Warning" badge to alert the officer

Test ID	Test Case Description	Pre-conditions	Test Steps	Expected Result
				of agent instability.
TC - UI-03	Payload Review Modal	Officer clicks on a parked payload in the queue.	Observe the modal details.	A detailed view opens, rendering the raw JSON payload in a readable format, alongside the originating agent_id and the intended destination URL.
TC - UI-04	Confirmation Dialogs & Optimistic UI	Officer clicks "Approve" on a payload.	Complete the confirmation dialog.	API call is dispatched. Upon a 200 OK response, the UI optimistically removes the item from the active queue and displays a success toast notification.

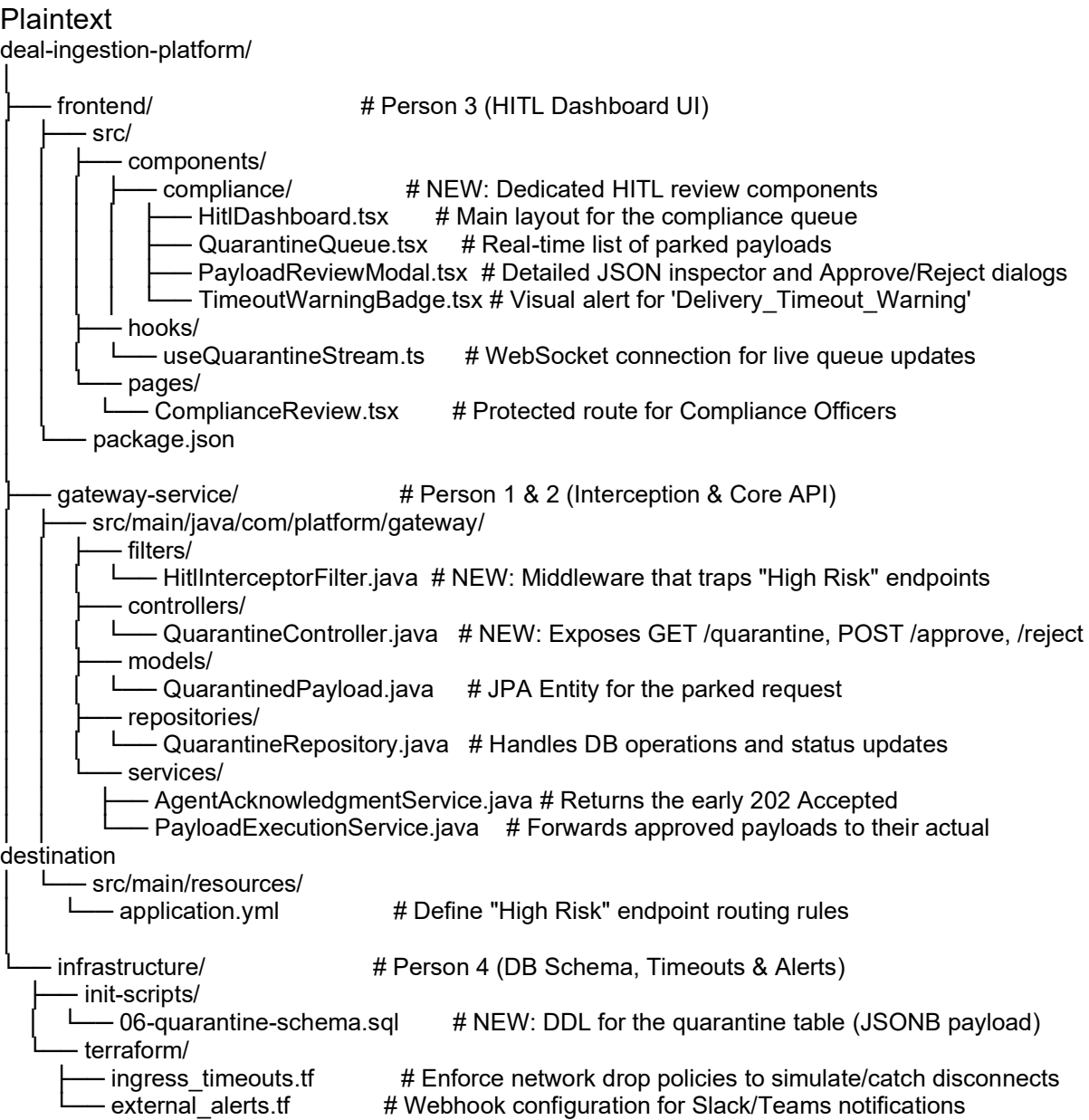
4. DevOps & Security Engineer Test Cases

Focus: External alerting, infrastructure timeouts, and zero-trust internal routing.

Test ID	Test Case Description	Pre-conditions	Test Steps	Expected Result
TC-SEC-01	Internal Routing Zero-Trust	Network is fully deployed.	Attempt to send a direct POST request to the destination /api/credit/approve endpoint from the AI Agent subnet (bypassing the Spring Boot Gateway).	Infrastructure firewall/ingress blocks the request (403 Forbidden or timeout).

Test ID	Test Case Description	Pre-conditions	Test Steps	Expected Result
				The destination endpoint only accepts traffic originating explicitly from the Gateway's internal IP.
TC-SE C-02	External Webhook Alerting	Notification bridge is configured for Slack/Teams.	Trigger a Delivery_Timeout_Warning state in the database.	The infrastructure automatically dispatches an alert to the designated compliance channel detailing the agent_id and the isolated payload ID.
TC-SE C-03	Gateway Timeout Simulation	Gateway read/write timeouts set to strict minimums.	AI Agent initiates a slow-loris or heavily delayed connection during payload delivery.	The ingress controller severs the connection when the threshold is hit, ensuring the Gateway doesn't hang indefinitely waiting on a dead agent.

Here is the recommended monorepo directory structure tailored specifically for **Sub-Story 3.1 (Middleware Interceptor & Quarantine State)** and **Sub-Story 3.2 (Parked Payload Execution & Rejection)**.



Key Component Roles

Service	Primary Addition	Rationale
Spring Gateway	HitlInterceptorFilter.java	Acts as the security checkpoint. It halts the standard routing chain for specific URLs, grabs the request body, and routes it to the database instead of the destination service.

Service	Primary Addition	Rationale
Spring Gateway	PayloadExecutionService.java	Houses the critical logic for Sub-Story 3.2a. When an officer clicks "Approve", this service reconstructs the exact original HTTP request from the database and fires it at the destination as if it came directly from the AI agent.
Infrastructure	06-quarantine-schema.sql	Creates the holding pen. Crucially, the payload column here should be implemented as a JSONB type in PostgreSQL to efficiently store and reconstruct complex, nested AI agent requests without string manipulation.
React UI	TimeoutWarningBadge.tsx	Conditionally renders only if the gateway reports a Delivery_Timeout_Warning, immediately drawing the compliance officer's eye to a potentially unstable autonomous agent that might try to execute the action a second time.